

00:00

So this is our lecture week 7, SIT112. In this lecture we're going to talk about data analysis. The objectives include melting the data in two or more columns, grouping and aggregating the data in a data frame, pivoting the data in a data frame or creating a pivot table, binning, selecting rows with the n largest values, ranking the rows, using pivot and pivot table methods, and other methods for specific analysis tasks. Before I talk about creating and plotting long data, I want to draw your attention to the fact that we have covered data cleaning, data preparation, and visualization. These different steps or processes of data analysis overlap. Sometimes data cleaning can be considered part of data preparation, and some data-preparation tasks can be considered part of analysis. There is no crisp boundary between these processes. Data cleaning and preparation significantly contribute to analysis. When we talk about data analysis specifically, we usually focus on the later stages, where we try to get insights from the data, draw hypotheses, and further study our findings. In its general sense, data analysis also includes data preparation and data cleaning to some extent. These definitions are conventions to help us understand what needs to be done and the sequence of tasks needed to perform data analysis.

03:47

Now let's look at creating and plotting long data. We have seen examples of long and wide data before. We use pivoting to create wide data and melting to create long data. Here we can look at examples of melting data to create long data and visualizing it. We have a cars data set, cars.pkl, a pickle file that contains a data frame with information about cars, including the price, curb weight, engine size, and car body. Each row represents a car and its corresponding attributes are captured by the columns. We are melting the cars data frame. Using pandas read_pickle returns a data frame from cars.pkl, reads the pickle file, creates a data frame, and names it cars. Then we use the pandas melt function. PD refers to the pandas handle. The first input is the data frame, cars. ID variables gets the value price, so price is used as the identifier. The value variables are the columns whose values are used for melting: engine size and curb weight. In the original data frame these are column names, but in the melted data frame they become values. The resulting data frame is in long form. We specify the variable name as feature and the value name as feature value.

08:32

The filtering statement shows part of the results. It filters the resulting data frame based on rows whose price is one of the specified values, extracting a subset corresponding to those prices. Price works as the identifier variable, because it was assigned to ID variables. Engine size and curb weight are selected as the value variables. These were columns in the original data frame, but their column titles are now used as values in the new data frame. They need a new title, feature, and their associated values are called feature value. Thus, feature can be engine size or curb weight, and feature value contains the corresponding number. The resulting data frame has a price column used as the identifier. For a particular price, there is one row for engine size and one row for curb weight, each with its corresponding value. This transforms, reshapes, or restructures the original data frame into long format using the pandas melt function.

12:47

We can visualize the melted or long-format data frame using the replot function of seaborn. By convention, SNS refers to an instance of seaborn, just as PD refers to pandas. We assume that seaborn has been imported as SNS. The data frame, cars melted, is provided as the input. We also specify the x- and y-axes. The x-axis is feature value, which can represent engine size or curb weight, and the y-axis is price. The feature column is specified as the column parameter, so two separate plots or subplots are created, one for each feature. In other words, one plot corresponds to curb weight and one corresponds to engine size. The x-axis for each subplot shows feature value and the y-axis shows price. Disabling the sharing of the x-axis gives each subplot its own x-axis.

17:44

Another way to visualize the long-format data is to combine the two subplots into one and use hue. Instead of using the column or feature parameter, we specify feature as hue. Different colors highlight the points corresponding to each feature value. Feature can again be engine size or curb weight; one color represents curb weight and another represents engine size. Instead of having two subplots, the two are combined into one, with colors distinguishing the points. The x-axis and y-axis remain feature value and price. These are two different ways of plotting melted data in long format.

19:39

The next topic is grouping and aggregating data. Calling the `groupby` function for a data frame returns a data-frame `groupby` object. We can work with this object. The `fires` data frame includes fire name, fire year, state, discovery date, containment date, acres burned, fire month, and days burning. We group the data frame by fire year and apply the `sum` function to the grouped data. `Sum` is applied to each group to calculate the sums of the numerical columns. Fire year becomes the index in the resulting data frame instead of the original numerical indices. The result contains the sum of acres burned, fire month, and days burning for each fire-year group. Each group corresponds to a year, and for each year we can see the total acres burned and the totals of the other numerical values. The exact meaning of fire month should be confirmed from the data-frame description, but the important point is that the value is the sum of the month values corresponding to that year.

24:21

Here is an example of grouping and applying a single aggregate method. We read the data into the `fires` data frame, reset the index, drop the old index, and display the first three rows. This time we group by state and calculate the mean of fire year, acres burned, fire month, and days burning for each state. Compared with the previous example, there is one more numerical column, fire year, because the previous grouping used fire year as the index and `sum` was applied only to acres burned, fire month, and days burning. `Mean`, like `sum`, is applied to numerical columns. Fire year is treated as a number, so its mean is also calculated for each state. Another single-aggregation example applies the `max` function while grouping by state, fire year, and fire month. The combination of these three columns specifies one group. We can think of this as three-stage grouping: first by state, then by fire year, and then by fire month. The `max` function finds the maximum discovery date, containment date, acres burned, and days burning for each group.

02:57

The `max` operation may produce a future warning. We need to be sure that the values for which we are finding a maximum are numbers, because the operation may lead to an error if the data types change in the future. The warning describes the internal mechanism of the `max` method and asks us to be careful. We can ignore it if we know that the result is valid and makes sense; otherwise, we should apply `max` only to co

lums for which finding a maximum is meaningful and whose values are numeric. A warning does not stop program execution and is not a runtime error. It warns about potential issues and asks us to be cautious because an operation that works now may cause problems in a future script or version.

31:57

In another example, we group the fires data frame by fire year. Setting `as_index=False` means that fire year does not become the new index. In effect, the grouped data frame is flattened by repeating fire-year values in the result. This is another way to achieve the effect of resetting the index. The resulting data frame has numerical indices starting from zero, while fire year is treated as a normal column. The sum function is then applied to all numerical columns of the yearly grouped data frame. One problem with this output is that the column titles are not descriptive: someone looking at the table cannot tell that the values represent sums of acres burned, fire month, or days burning. We can use the `rename` function after grouping and aggregation, providing the original column names and their new, more meaningful names.

35:27

Here is an example of multiple aggregation. We group the data by state, fire year, and fire month, and then apply the three functions `sum`, `count`, and `mean` to the grouped data frame. The grouped frame contains monthly information about fires. The aggregations are applied to numerical columns. The result contains `sum`, `count`, and `mean` for acres burned, and `sum`, `count`, and `mean` for days burning. For every state, fire year, and fire month group, we obtain the sum of acres burned, the number of values or fires, and the mean of acres burned. The same three calculations are performed for days burning. `Sum` adds all values in a group, `count` tells us how many values are in the group, and `mean` adds the values and divides by the count.

40:10

Another multiple-aggregation example selects only the days-burning column and applies `sum`, `count`, and `mean` to that column. This differs from the previous example, where the same aggregations were applied to two numerical columns. We can extend the explanation to each aggregation column. A further example applies different aggregations to different columns by passing a dictionary to the aggregation function. The dictionary contains column names as keys and lists of functions as values. We select

ct acres burned, days burning, and fire name. Sum, max, and mean are applied to acres burned; sum and mean are applied to days burning; and count is applied to fire name. Fire name is a string, but we can still count how many strings occur in each group. The resulting output contains the requested aggregation values for each group.

43:58

The next topic is creating and using pivot tables. In the example, top states is initially just the grouped data frame, not yet a list of top states. It contains fires grouped by state and fire year, with sum applied to the numerical columns. We then use the query function to filter the grouped data frame so that it includes only states contained in the specified list, and display the first two rows. No pivoting has occurred yet. We apply the pivot function to the filtered data frame, setting the index to fire year and specifying the columns using state. The selected states become labels of the columns, and the resulting data frame is in wide format. The value for each year-and-state pair is acres burned, representing the sum of acres burned for the data points corresponding to that year and state. Calling plot on the resulting wide data frame creates a line plot, with one line for each state.

04:45

The pivot_table function provides more flexibility than pivot because it can both reshape the data frame and apply an aggregation function. Here we apply mean. The result is similar to pivot: we specify the index, specify the columns as states, filter the data frame based on the top states, and use acres burned as the values. The mean is calculated for acres burned for each year and its corresponding state. The result can again be visualized with plot, producing a line for the mean acres burned for each state. A useful question is to compare pivot and pivot_table. We briefly explained the difference, but you can spend more time understanding it by asking for additional examples and seeing how the two functions work in different situations.

49:40

Before discussing bins, I am going to take a five-minute break so we can rest, and then we will continue. The first example creates bins of equal size. We work with the fires data set and use a query to extract fires from 2010 where days burning is greater than zero. We then drop rows with null values and store the result in fires_filtered. The acres-burned values are divided into equal-size bins using the pandas cut function.

Setting bins equal to four creates four bins of equal size based on the values in acres burned. Each index receives the bin corresponding to the range in which its value falls. The displayed rows may all show the same bin, but the full result contains all four bins. The intervals represent the first, second, third, and fourth bins, and the bins are equal in size.

53:28

For example, a value such as 16.8 falls into the range that contains it, and its corresponding bin is assigned. We can also create custom bins. Instead of specifying the number of bins, we provide the points where the bins start and end. In the example, the list starts at zero and goes to 100,000, then to 200,000, 300,000, and 400,000. The ranges specify the four bins. The interval convention determines which boundary is included; the end of the range is inclusive in the example, while the starting boundary is not. The first range is 0 to 100,000, the second is 100,000 to 200,000, the third is 200,000 to 300,000, and the fourth is 300,000 to 400,000. We can assign labels such as small, medium, large, and very large. The labels are ordinal: small is less than medium, medium is less than large, and large is less than very large. Labels are more readable than the actual interval values.

58:38

We can use `value_counts` to find how many values fall into each category or bin. In the example, 1,855 data points fall into the small category, two fall into medium, one falls into very large, and none fall into large. The bins may therefore be unbalanced, which may not be ideal for certain machine-learning tasks or analyses. This leads to creating bins with equally or almost equally distributed data points. We can use `qcut`, which is similar to `cut` but takes the distribution of the data into account and creates quartiles. Instead of dividing the numerical range into equal-width bins, `qcut` aims to place almost equal numbers of data points in each bin. The widths of the bins do not need to be equal. We specify the `acres-burned` column, request four bins, and provide labels small, medium, large, and very large. The resulting value counts are approximately balanced: 466 in small, 465 in medium, 464 in very large, and 463 in large. This is desirable for some machine-learning tasks.

01:04:34

There is a short video showing how to use `qcut`, with a couple of examples, but we w

ill move forward with the slides. After creating bins, we can plot the binned data. Seaborn's `catplot` helps visualize categorical data; `cat` stands for category. We provide the filtered fires data frame, set `kind` to `count`, and use `fire month` on the x-axis. The `hue` specifies fire size, which can be small, medium, large, or very large. Fire size is the new column containing the bin labels created earlier. The plot therefore shows the count of rows in the filtered data frame corresponding to each fire-size category for each fire month. The graph displays the number of data points in each bin.

01:07:41

More useful skills for data analysis include selecting the `n` largest rows. The `nlargest` function finds the top `n` rows from a data frame based on a particular column or criterion. Using engine size as the criterion lets us find the top six rows with the highest engine-size values. We can also use two columns, engine size and price, to find the top six data points. The comparison first uses engine size; if two rows are tied, price is used to distinguish them. A row with a greater engine size is ranked higher, and if the engine sizes are equal, the row with the higher price is ranked higher.

01:10:53

We can calculate percentage changes using `pct_change`. For each row, the function takes the value in that row, subtracts the corresponding value from the previous row in the same column, and divides by the previous value. This gives the percentage change. The first row receives `NaN` because there is no preceding row, so the calculation cannot be performed. Even treating the missing previous value as zero would result in division by zero. Therefore, the percentage change cannot be calculated for the first row.

01:13:55

We can also rank rows using the `rank` function. After grouping and calculating sums, we extract acres burned and days burning, and then rank the rows. Setting `ascending=False` ranks the values in descending order. The ranks are stored in a new column, such as `acres_rank`, and the rows are ranked based on acres burned. The row with the highest acres-burned value receives rank one. Other rows receive ranks according to their positions in the full data frame, even though only the first few rows may be displayed. Scientific notation can make a number look smaller while its exponent makes the actual number larger. The rank column shows the resulting rank for each row.

01:18:05

The method parameter in rank specifies how ties are handled. If two or more rows have the same value, we need to decide whether they receive equal ranks. With method='max', the largest rank that would have been assigned to the tied values is assigned to all of them. For example, if two rows would otherwise receive ranks two and three, both receive rank three, so no row receives rank two. This gives flexibility in handling tied values. The choice depends on what you want to do with the data. This wraps up today's lecture, and I am going to stop at this point.